

Lab Session 2: State estimation

Laboratory on Least Mean Square position and bias estimation | **LAB BRIEFING**

Andrea Nardin, Simone Zocca, prof. Fabio Dovis
name.surname@polito.it



Estimating Position from GNSS observables



Radionavigation in GNSS is based on a set of measurements that a receiver retrieves by receiving and processing satellites navigation **signals**.

These measurements are often referred to as **observables** and include **pseudorange** and **pseudorange-rate** measurements, **Doppler shift** measurements and others for each “visible” satellite (as in Android raw measurements, see Lab Session 1)

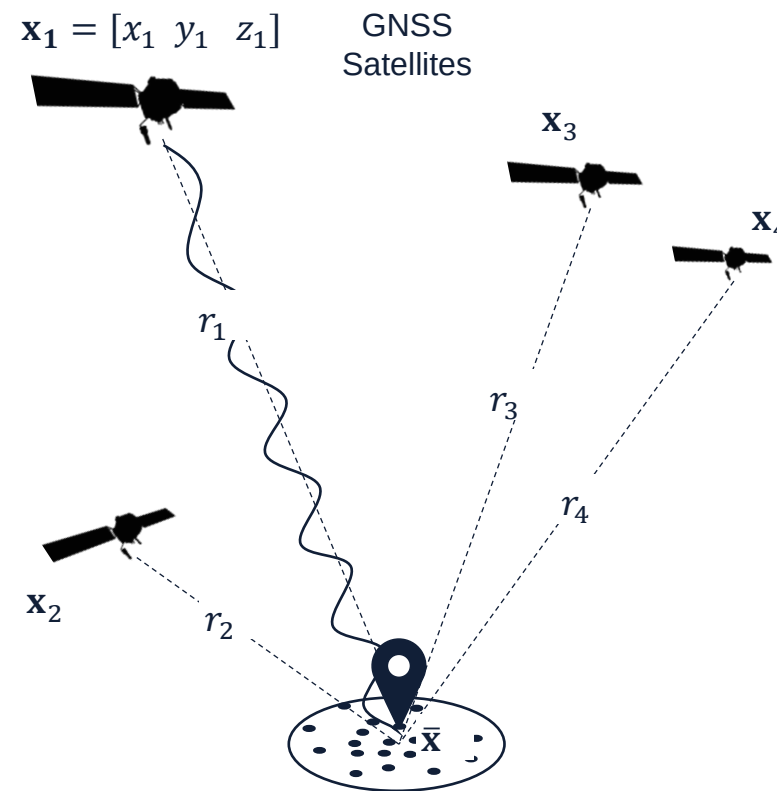
We know the **observables** depend on the actual **state** of the receiver. By inverting known observable-state relationships, through a sufficient amount of **observables** we can infer the **receiver state**.

Example (pseudorange measurements)

$$\rho_1 = \sqrt{(x_1 - x)^2 + (y_1 - y)^2 + (z_1 - z)^2} + b_u$$

Measured
Unknown
Known A-priori
Unknown
Known A-priori
Unknown
Known A-priori
Unknown

How to estimate the unknowns from observables ?



Snapshot at time instant n of the radionavigation problem for a GNSS receiver located at $\bar{x}_n$ and a set of GNSS satellites whose signals can be received and tracked by the receiver.

Estimating Position from GNSS observables



Radionavigation in GNSS is based on a set of measurements that a receiver retrieves by receiving and processing satellites navigation **signals**.

These measurements are often referred to as **observables** and include **pseudorange** and **pseudorange-rate** measurements, **Doppler shift** measurements and others for each “visible” satellite (as in Android raw measurements, see Lab Session 1)

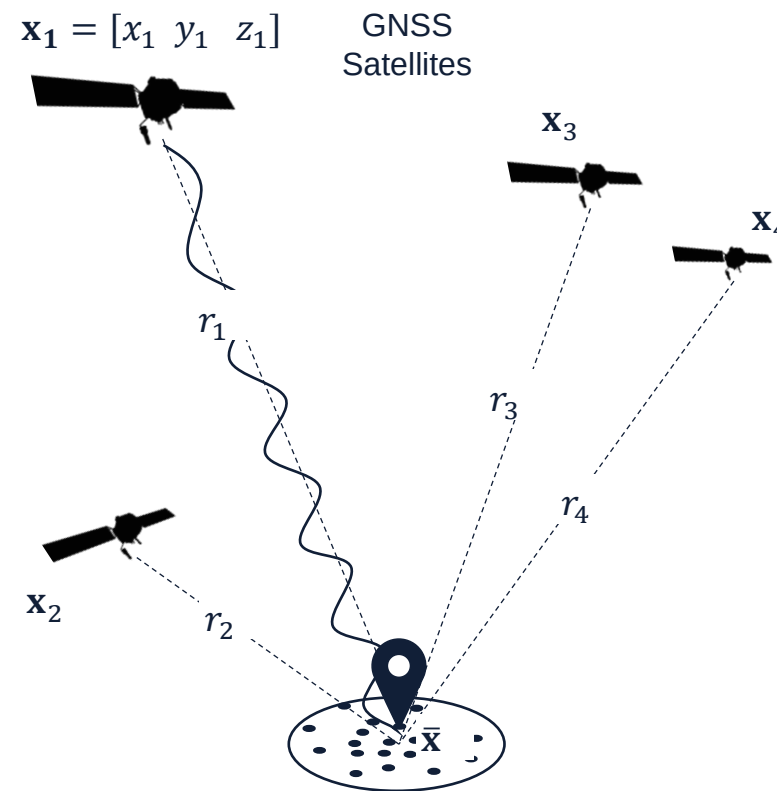
We know the **observables** depend on the actual **state** of the receiver. By inverting known observable-state relationships, through a sufficient amount of **observables** we can infer the **receiver state**.

Example (pseudorange measurements)

$$\begin{array}{cc} \text{Position} & \text{State} \\ \bar{\mathbf{x}} = [x \ y \ z] & \mathbf{x} = [\bar{\mathbf{x}} \ b_u] \end{array}$$

$$\underbrace{\rho_1}_{\text{Measured}} = \underbrace{\|\bar{\mathbf{x}} - \mathbf{x}_1\|}_{\text{Unknown}} + \underbrace{b_u}_{\text{Known A-priori}}$$

How to estimate the unknowns from observables ?



Snapshot at time instant n of the radionavigation problem for a GNSS receiver located at $\mathbf{x}_A$ and a set of GNSS satellites whose signals can be received and tracked by the receiver.

Least Mean Square applied to Single Point Positioning (SPP)



- Under some assumptions, the **Position, Velocity and Time** (PVT) solution is estimated by linearizing a multilateration problem through a **Least Mean Square** (LMS) algorithm (a.k.a. LS)

$$\Delta \mathbf{x} = (\mathbf{H}^T \mathbf{H})^{-1} \mathbf{H}^T \Delta \boldsymbol{\rho}$$

- One possible way to solve for the PVT is to **recursively** apply the LMS algorithm by updating the **linearization point** at each iteration to obtain a more accurate solution
- At each iteration, the estimated PVT is expected to be closer to the actual user location

Least Mean Square applied to Single Point Positioning (SPP)



- Under some assumptions, the **Position, Velocity and Time** (PVT) solution is estimated by linearizing a multilateration problem through a **Least Mean Square** (LMS) algorithm (a.k.a. LS)

$$\Delta \mathbf{x}^k = \left((\mathbf{H}^k)^T \mathbf{H}^k \right)^{-1} (\mathbf{H}^k)^T \Delta \boldsymbol{\rho}^k$$

- One possible way to solve for the PVT is to **recursively** apply the LMS algorithm by updating the **linearization point** at **each iteration k** to obtain a more accurate solution
- At **each iteration**, the estimated PVT is expected to be closer to the actual user location

Least Mean Square applied to Single Point Positioning (SPP)



- Under some assumptions, the **Position, Velocity and Time** (PVT) solution is estimated by linearizing a multilateration problem through a **Least Mean Square** (LMS) algorithm (a.k.a. LS)

$$\Delta \mathbf{x}_n^k = \left((\mathbf{H}_n^k)^T \mathbf{H}_n^k \right)^{-1} (\mathbf{H}_n^k)^T \Delta \boldsymbol{\rho}_n^k$$

- One possible way to solve for the PVT is to **recursively** apply the LMS algorithm by updating the **linearization point** at **each iteration k** to obtain a more accurate solution
- At **each iteration**, the estimated PVT is expected to be closer to the actual user location
- At **each epoch n** , new measurements are processed and the **iterative LMS** restarts
- In this lab we will process pseudorange to solve for position and clock bias only.
Velocity estimation requires pseudorange rate measurements (not available in this lab).

Recursive Least Mean Square

- For each time instant n , a number of iterations $k = 1, \dots, K$ is performed to process the same measurements vector $\boldsymbol{\rho}_n$

Initialization	$\hat{\mathbf{x}}_n^0$	Initial position/linearization point
		Nominal pseudoranges from $\hat{\mathbf{x}}_n^0$ and satellite positions
		$\hat{\rho}_{j,n} = \sqrt{(x_{j,n} - \hat{x}_n^k)^2 + (y_{j,n} - \hat{y}_n^k)^2 + (z_{j,n} - \hat{z}_n^k)^2} + b_{u,n}^k$
Procedure for k -th iteration	$\Delta \hat{\boldsymbol{\rho}}_n^k = \hat{\boldsymbol{\rho}}_n^k - \boldsymbol{\rho}_n$	Measured/Observed pseudorange measurements
	$\Delta \hat{\mathbf{x}}_n^k = ((\mathbf{H}_n^k)^T \mathbf{H}_n^k)^{-1} (\mathbf{H}_n^k)^T \Delta \hat{\boldsymbol{\rho}}_n^k$	
	$\hat{\mathbf{x}}_n^k = \hat{\mathbf{x}}_n^{k-1} + \Delta \hat{\mathbf{x}}_n^k$	

- $\mathbf{H}$ and $\hat{\mathbf{x}}_n^k$ are updated at each iteration k
- The **initial** linearization point, $\hat{\mathbf{x}}_n^0$, could be the same at each time n (but why should be?)
- K has to be chosen in order to obtain a “stable” solution (generally $K < 10$)

Recursive Least Mean Square

- $\hat{\mathbf{x}}_n^k = [\hat{x}_n^k \quad \hat{y}_n^k \quad \hat{z}_n^k \quad \hat{b}_n^k]$ is the **state estimate** (Position and Time) at the k -th iteration
- The **number of visible satellites** J , is **not constant** over the observation timespan
- $\Delta \mathbf{p}_n$ is the vector of the J_n **pseudorange differences** (between observed/measured and calculated w.r.t. the linearization point) at the k -th iteration
- At each iteration k , $\mathbf{H}_n^k$ is the geometrical matrix obtained from the previous estimated position:

$$\mathbf{H}_n^k = \begin{bmatrix} a_{x,1n}^k & a_{y,1n}^k & a_{z,1n}^k & 1 \\ a_{x,2n}^k & a_{y,2n}^k & a_{z,2n}^k & 1 \\ a_{x,3n}^k & a_{y,3n}^k & a_{z,3n}^k & 1 \\ \vdots & \vdots & \vdots & \vdots \\ a_{x,Jn}^k & a_{y,Jn}^k & a_{z,Jn}^k & 1 \end{bmatrix}$$

Satellite coordinates are kept fixed for any k

$$a_{x,jn}^k = \left[\frac{x_{j,n} - \hat{x}_n^k}{\hat{r}_{j,n}} \right], a_{y,jn} = \dots, a_{z,jn} = \dots$$

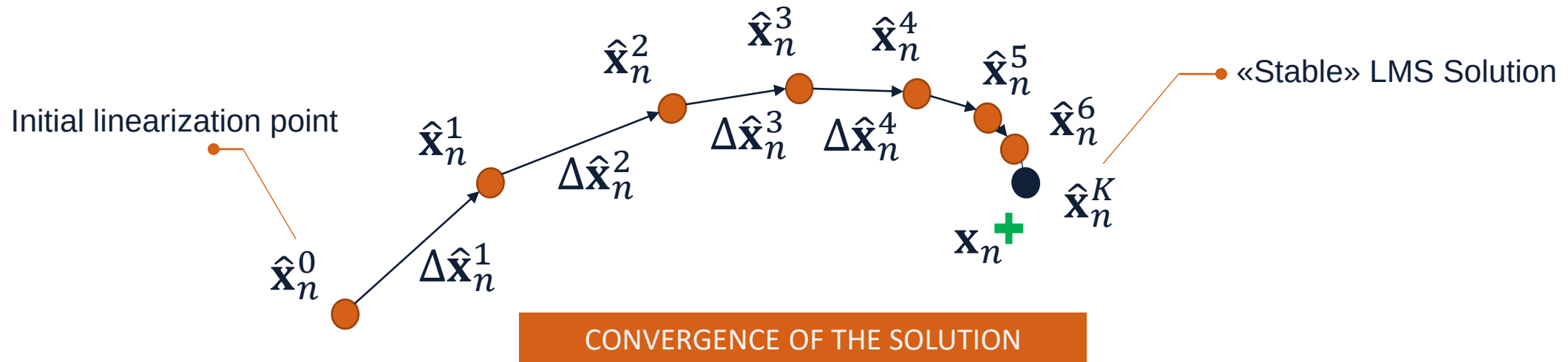
$$\hat{r}_{j,n} = \sqrt{(x_{j,n} - \hat{x}_n^k)^2 + (y_{j,n} - \hat{y}_n^k)^2 + (z_{j,n} - \hat{z}_n^k)^2}$$

- Geometrical range from $\hat{\mathbf{x}}_n^k$ and satellite position ($\neq \hat{\rho}_{j,n}$)

Recursive Least Mean Square

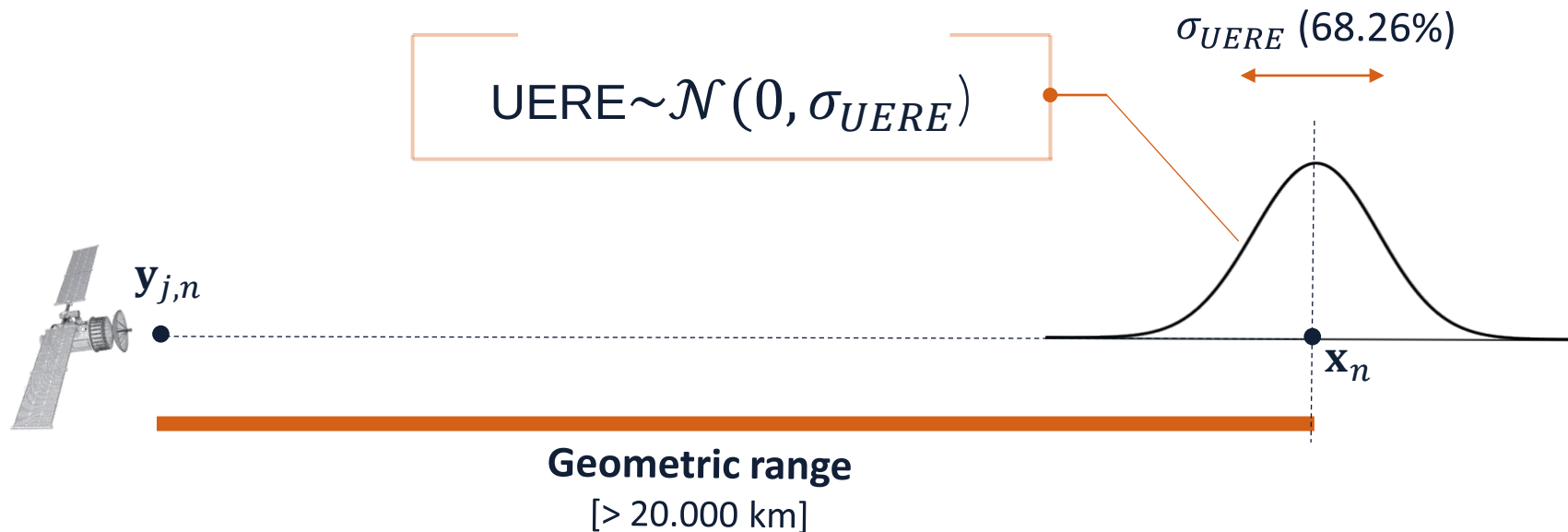
- By adding $\Delta\hat{\mathbf{x}}_n^k$ to the estimated position at the previous iteration, the position estimate is updated, and a new solution is found.

$$\hat{\mathbf{x}}_n^k = \hat{\mathbf{x}}_n^{k-1} + \Delta\hat{\mathbf{x}}_n^k$$



From raw to corrected pseudorange measurements

- In real applications, the measurement of the pseudoranges must be **corrected** of all the predictable error contributions (bias terms).
- After the correction, a **random contribution** is left that is modeled as *User Equivalent Range Error* (UERE). This error along with the geometrical dilution of precision are responsible for the **state estimation error** and condition the precision of the solution.



Different UERE for different satellites

A naive LMS algorithm attributes the **same relevance to all the measurements** but actual pseudorange measurements are **not equally precise**.

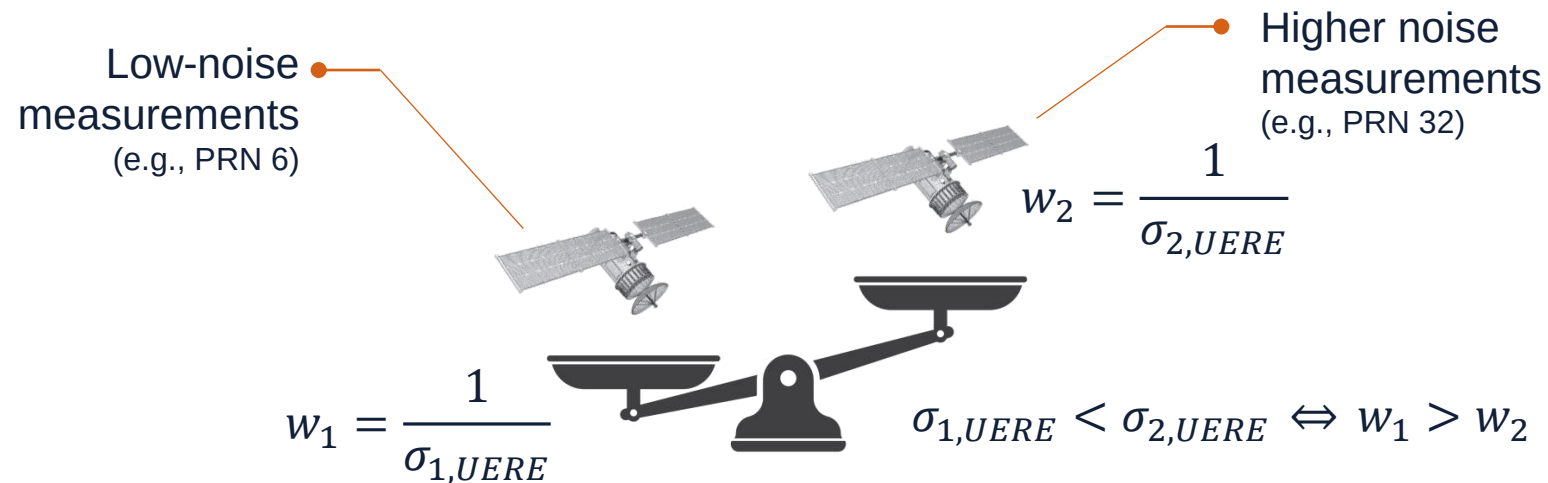
- In **real cases**, each pseudorange may be characterized by a different value of standard deviation, $\sigma_{j,UERE}$
- If pseudorange errors are assumed uncorrelated, their covariance matrix is written as:

Measurements Error Covariance

$$\mathbf{R} = \text{Cov}(\boldsymbol{\rho}) = \begin{bmatrix} \sigma_{1,UERE}^2 & 0 & 0 & \dots & 0 \\ 0 & \sigma_{2,UERE}^2 & 0 & \dots & 0 \\ 0 & 0 & \sigma_{3,UERE}^2 & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \dots & \sigma_{J,UERE}^2 \end{bmatrix}$$

Weighted Least Mean Square (WLMS/WLS)

- A more general approach foresees to solve the system by means of a **Weighted Least Mean Squares, or simply Weighted Least Squares** (WLS) algorithm, characterized by the assumption of a non-unitary covariance matrix **R**
- Since some measurements may be more accurate than others, the measurement **accuracy is different** for each measurement
- WLS uses a weighting matrix **W** = **R**⁻¹ to give to the most uncertain measurements the smallest weight.



Uncorrelated measurements

- For each time instant n , a number of iterations $k = 1, \dots, K$ is performed for the same measurements vector $\boldsymbol{\rho}_n$
- The WLS algorithm is a generalization of the LMS algorithm
- The weight matrix can be estimated from the measurements

Initialization	$\hat{\mathbf{x}}_n^0$
Procedure for k -th iteration	$\Delta \hat{\boldsymbol{\rho}}_n^k = \hat{\boldsymbol{\rho}}_n^k - \boldsymbol{\rho}_n$
	$\bar{\mathbf{H}}_n^k = \left((\mathbf{H}_n^k)^T \mathbf{W} \mathbf{H}_n^k \right)^{-1} (\mathbf{H}_n^k)^T \mathbf{W}$
	$\Delta \hat{\mathbf{x}}_n^k = \bar{\mathbf{H}}_n^k \Delta \hat{\boldsymbol{\rho}}_n^k$
	$\hat{\mathbf{x}}_n^k = \hat{\mathbf{x}}_n^{k-1} + \Delta \hat{\mathbf{x}}_n^k$

• Weighting Matrix, $\mathbf{W} = \mathbf{R}^{-1}$

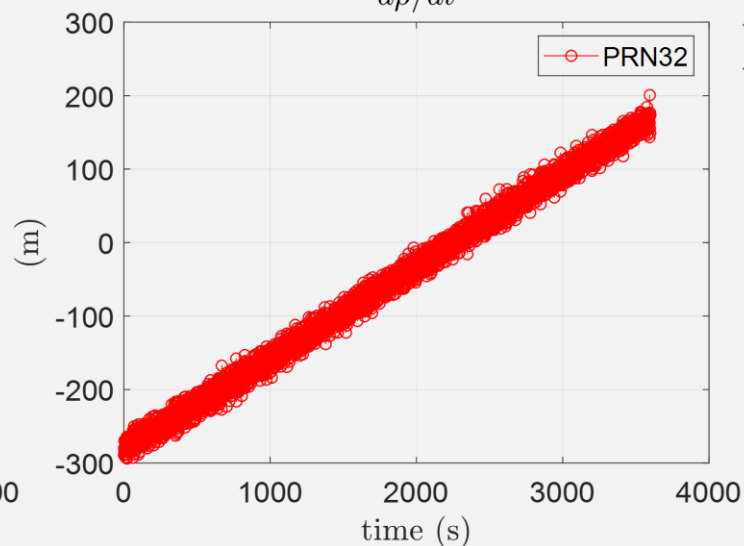
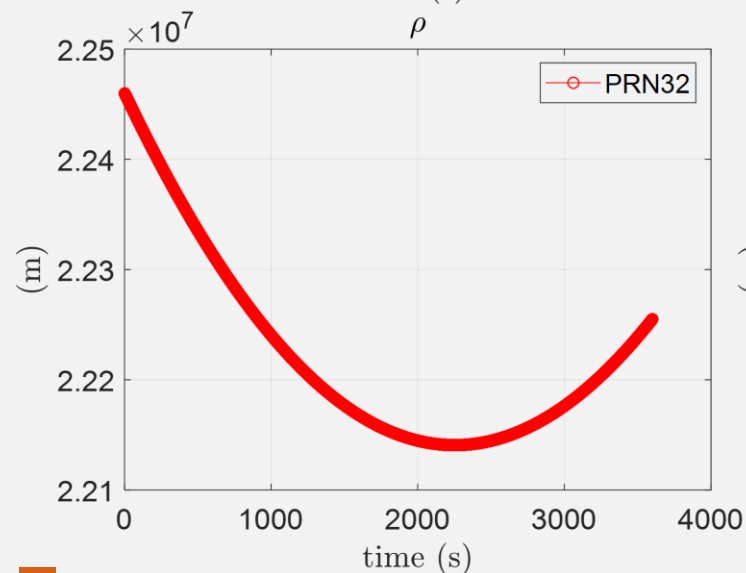
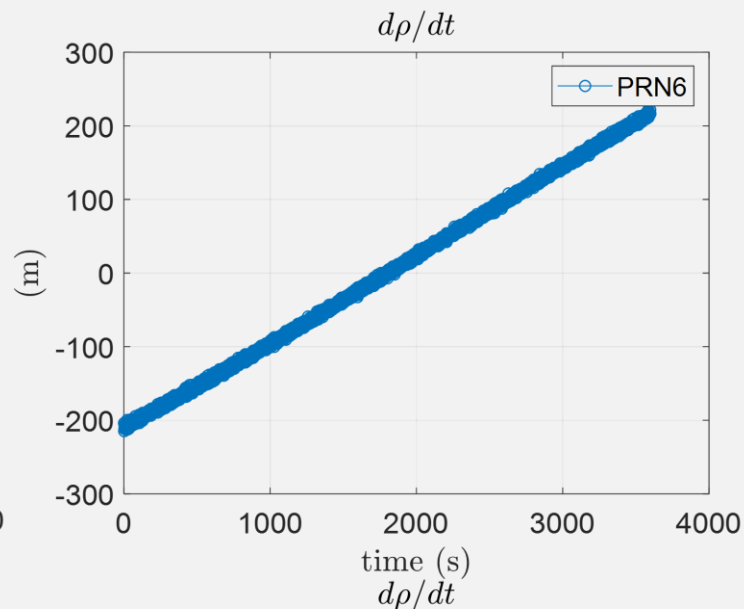
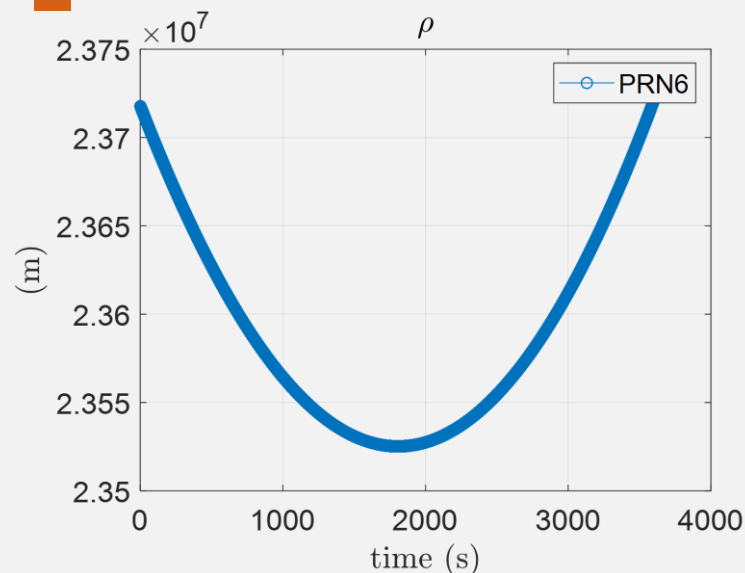
Different errors for different satellites

- The direct estimation of the measurements error covariance matrix is not possible since each $\rho_{j,n}$ is **time-dependent** (multiple realizations for the same instant n are not available)
- We can estimate the $\sigma_{j,\text{UERE}}$ for each pseudorange, ρ_j , **analyzing the error** on the pseudorange itself **along the time**
- **Dependency** on time must be **removed** (measurement de-trending)
- Since pseudorange measurement follows a nearly quadratic trend, the **second order derivative** is sufficient to remove their trend along the time

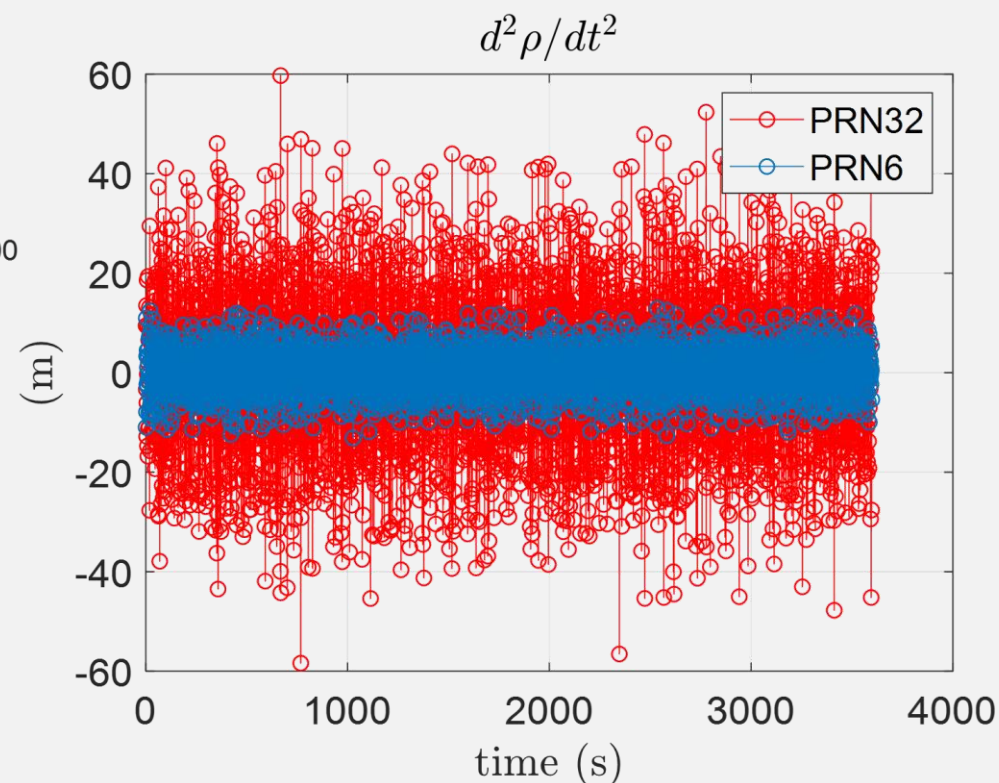


Use MATLAB function `diff(X, order)`

Pseudorange de-trending by differentiation



Detrended time series allows to observe the different UEREs affecting the pseudoranges measured via different signals



Lab Session 2: State estimation

Laboratory on Least Mean Square position and bias estimation | DATASETS AND TASKS

Andrea Nardin, Simone Zocca, prof. Fabio Dovis
name.surname@polito.it



Variables for Galileo constellation | Single Epoch



USER and SATELLITE STATE VECTORS (position and clock bias)

Unknown User Position

$$\mathbf{x} = [x \quad y \quad z \quad b_{GAL}]$$

Known Satellite Position

$$\mathbf{y}_j = [x_j \quad y_j \quad z_j]$$

GAL_SAT_POS.mat

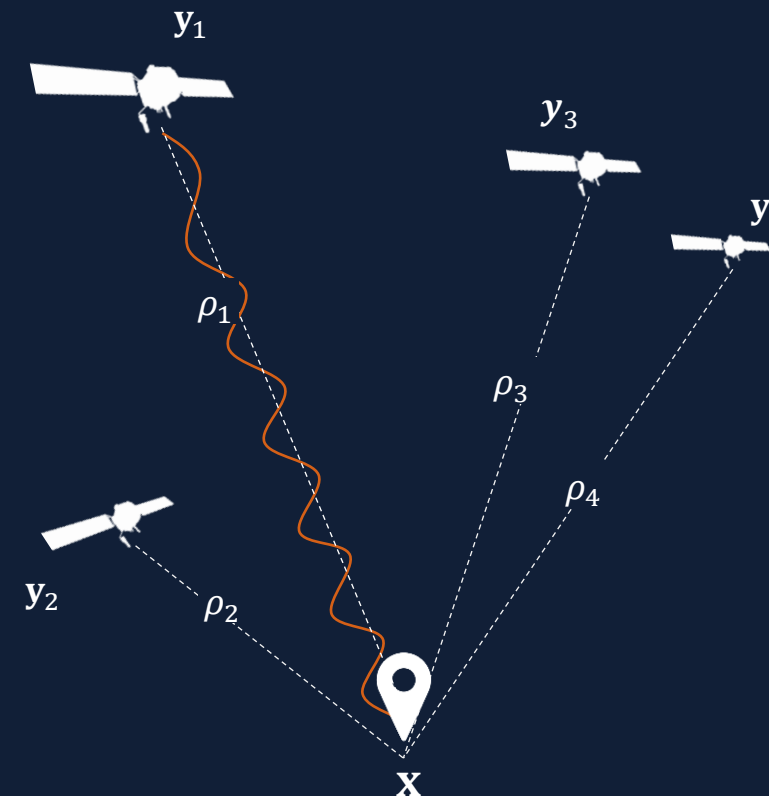
PSEUDORANGE EQUATION

Measured range distance from the satellite

$$\rho_j = \rho(\mathbf{x}, \mathbf{y}_j) = \sqrt{(x - x_j)^2 + (y - y_j)^2 + (z - z_j)^2} + b_{GAL}$$

PSEUDORANGE.mat

- $j \in (1, 2, \dots, J)$ is the **satellite identifier**
- The demo data collection contains one epoch of observables
- The unknown location coordinates should be Lat. -33.9248, Lon. 18.4241
Gooseberry, Cape Town, 9100, South Africa



Variables for GPS constellation | Multi-epoch



USER and SATELLITE STATE VECTORS (position and clock bias)

Unknown User Position

$$\mathbf{x}_n = [x_n \quad y_n \quad z_n \quad b_{n,GPS}]$$

Known Satellite Position

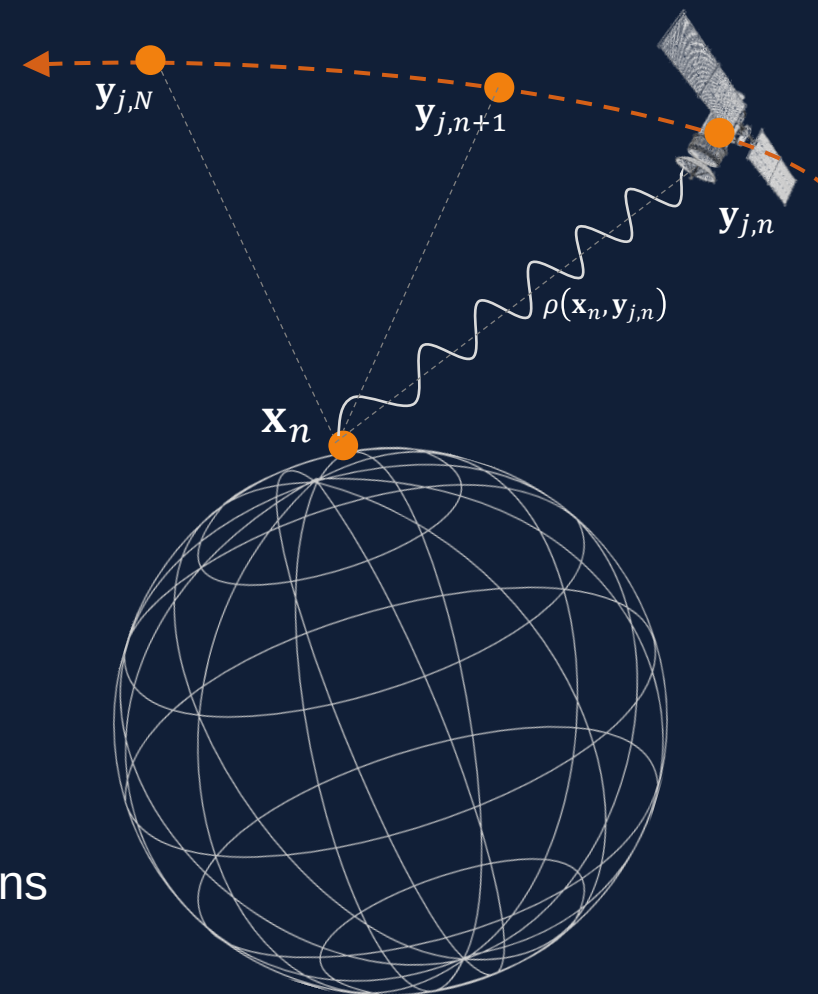
$$\mathbf{y}_{j,n} = [x_{j,n} \quad y_{j,n} \quad z_{j,n}]$$

PSEUDORANGE EQUATION

Measured range distance from the satellite

$$\rho_{j_n} = \rho(\mathbf{x}_n, \mathbf{y}_{j,n}) = \sqrt{(x_n - x_{j,n})^2 + (y_n - y_{j,n})^2 + (z_n - z_{j,n})^2} + b_{n,GPS}$$

- $n \in (1, 2, \dots, N)$ is the **time index**
- $j \in (1, 2, \dots, J)$ is the **satellite identifier**
- Data collections include **3600 seconds** (1 hour) of satellites observations from a static position $\mathbf{x}_n: \mathbf{x}_n = \mathbf{x}_{n+1} = \dots = \mathbf{x}_N$, where $N = 3600$



Observables Data Structures



PSEUDORANGE MEASUREMENTS (GPS)

RHO.GPS				
	$n = 1$	$n = 2$	..	$n = N$
GPS.PRN_1	$\rho(\mathbf{x}_1, \mathbf{y}_{1,1})$	$\rho(\mathbf{x}_2, \mathbf{y}_{1,2})$	...	$\rho(\mathbf{x}_N, \mathbf{y}_{1,N})$
GPS.PRN_2	$\rho(\mathbf{x}_1, \mathbf{y}_{2,1})$	$\rho(\mathbf{x}_2, \mathbf{y}_{2,2})$	...	$\rho(\mathbf{x}_N, \mathbf{y}_{2,N})$
GPS.PRN_3	$\rho(\mathbf{x}_1, \mathbf{y}_{3,1})$	$\rho(\mathbf{x}_2, \mathbf{y}_{3,2})$	...	$\rho(\mathbf{x}_N, \mathbf{y}_{3,N})$
...	...	...	...	...
GPS.PRN_J	$\rho(\mathbf{x}_1, \mathbf{y}_{J,1})$	$\rho(\mathbf{x}_2, \mathbf{y}_{J,2})$	...	$\rho(\mathbf{x}_N, \mathbf{y}_{J,N})$

SATELLITE POSITIONS FROM EPHEMERIS (GPS)

SV_ECEF.GPS		Ex: SV Position State Vector History			
SV ID	$\mathbf{Y}_j$	$\mathbf{Y}_1$	x	y	z
GPS.PRN_1	$\mathbf{Y}_1$	$n = 1$	$x_{1,1}$	$y_{1,1}$	$z_{1,1}$
GPS.PRN_2	$\mathbf{Y}_2$	$n = 2$	$x_{1,2}$	$y_{1,2}$	$z_{1,2}$
GPS.PRN_3	$\mathbf{Y}_3$	$n = 3$	$x_{1,3}$	$y_{1,3}$	$z_{1,3}$
...	...	...	...	...	...
GPS.PRN_J	$\mathbf{Y}_J$	$n = N$	$x_{1,N}$	$y_{1,N}$	$z_{1,N}$

- The number of visible satellites is not constant over observation time n
- The folder named «NominalUERE» contains pseudorange measurements with the same σ_{UERE}
- The folder named «RealisticUERE» contains pseudorange measurements with satellite-dependent σ_{UERE}
- Pseudorange measurements can be considered an ergodic random process over short time periods
- It is possible to select different constellations using strings GPS, GLO, BEI, GAL in the data structure

A. State estimation | Single-epoch

TASK

A.1

Load data from the *single-epoch* folder and verify the number of available pseudorange measurements in the data array `PSEUDORANGE.mat`.

TASK

A.2

Extract the satellites positions associated to the observables

TASK

A.3

Implement a recursive least square algorithm to estimate the position and clock bias states of the receiver and find its geodetic coordinates (i.e., Latitude, Longitude)

TASK

A.4

By using the `geoplot` built-in function of MATLAB, generate a figure showing the estimated location on a map.

While executing all the tasks, try to design your routines as MATLAB **functions** to create your own library. The same function can be used in the next task, without re-coding from scratch.

B. Lab session | Multi-epoch



TASK

B.1

Load data from the *NominalUERE* folder. Check the satellites **visibility** at each time instant n , for all the constellations. Plot the number of the **visible satellites** as well as the measured pseudoranges along the time.

TASK

B.2

Choose a dataset and a constellation (e.g., GPS, Galileo) and implement a Least Mean Square (LMS) positioning algorithm to estimate the user state, $\hat{\mathbf{x}}_n^k$, at each time instant n . Convert the estimated solution from ECEF to LLA coordinates and verify the position on Google Earth by means of *writeKML_GoogleEarth.m* (provided in *Utilities*) or MATLAB's *geoplot()*.

TASK

B.3

a) Compute the average position; b) compute the *position error* for each time instant with respect to the average position; c) compute the *standard deviation of the position error* and d) compare the quality of the obtained solution for different datasets and constellations. Motivate the results.

C. Lab session | Multi-epoch, Weighted LMS



TASK

C.1

Estimate the $\sigma_{j, \text{UERE}}$ for all satellites using the dataset from the *realisticUERE* folder.

TASK

C.2

Implement the **Weighted Least Mean Square** (WLMS) and repeat Tasks B.2 and B.3.

TASK

C.3

Compare the performance of LMS and WLMS on *realisticUERE* dataset.

Simulation hint (LMS/WLMS implementation)

For each epoch $n = 1:N$

- Find the visible pseudorange measurements
- Retrieve the corresponding satellites coordinates $\mathbf{y}_{j,n}$
- Build the H matrix
- Build the measurement vector $\boldsymbol{\rho}_n = [\rho_{1,n} \quad \dots \quad \rho_{J,n}]$
- Compute PVT solution (K iterations)

end

Number of LMS/WLMS iterations

Use a «reasonable» number of iterations, (i.e. $K < 10$)